

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Assignment 3

Submitted by

KETANPREET

SINGH(102303386)

Submitted to

SAIF NALBAND

Question 1: Assignment Questions:

Use make file and implement. You are given m input vectors, each with n numbers. Your task is to calculate the correlation between every pair of input vectors.

Interface

You need to implement the following function:

```
void correlate(int ny, int nx, const float* data, float* result)
```

Here data is a pointer to the input matrix, with ny rows and nx columns. For all $0 \leq y < ny$ and $0 \leq x < nx$, the element at row y and column x is stored in $data[x + y*nx]$. The function has to solve the following task: for all i and j with $0 \leq j < i < ny$, calculate the correlation coefficient between row i of the input matrix and row j of the input matrix, and store the result in $result[i + j*ny]$.

Make sure there is main.cpp correlate.cpp and makefile

1. Implement a simple sequential baseline solution. Do not try to use any form of parallelism yet; try to make it work correctly first. Please do all arithmetic with double precision floating point numbers.
2. Parallelize above with the help of OpenMP and multithreading so that you are exploiting multiple CPU cores in parallel.

3. Using all resources that you have in the CPU, solve the task as fast as possible. You are encouraged to exploit instruction-level parallelism, multithreading, and vector instructions whenever possible, and also to optimize the memory access pattern.
4. Give the size of the matrix from the command line and use perf stats to evaluate the program for sequential and parallel. Increase the number of threads and size of matrix.

Solution

Folder Structure



FOR Compile :

USE MAKE

Usage: ./program <matrix_size> <seq-0 OR PAR-1>
perf stat ./program 2000 0

Performance Analysis FOR :

./program 2000 1

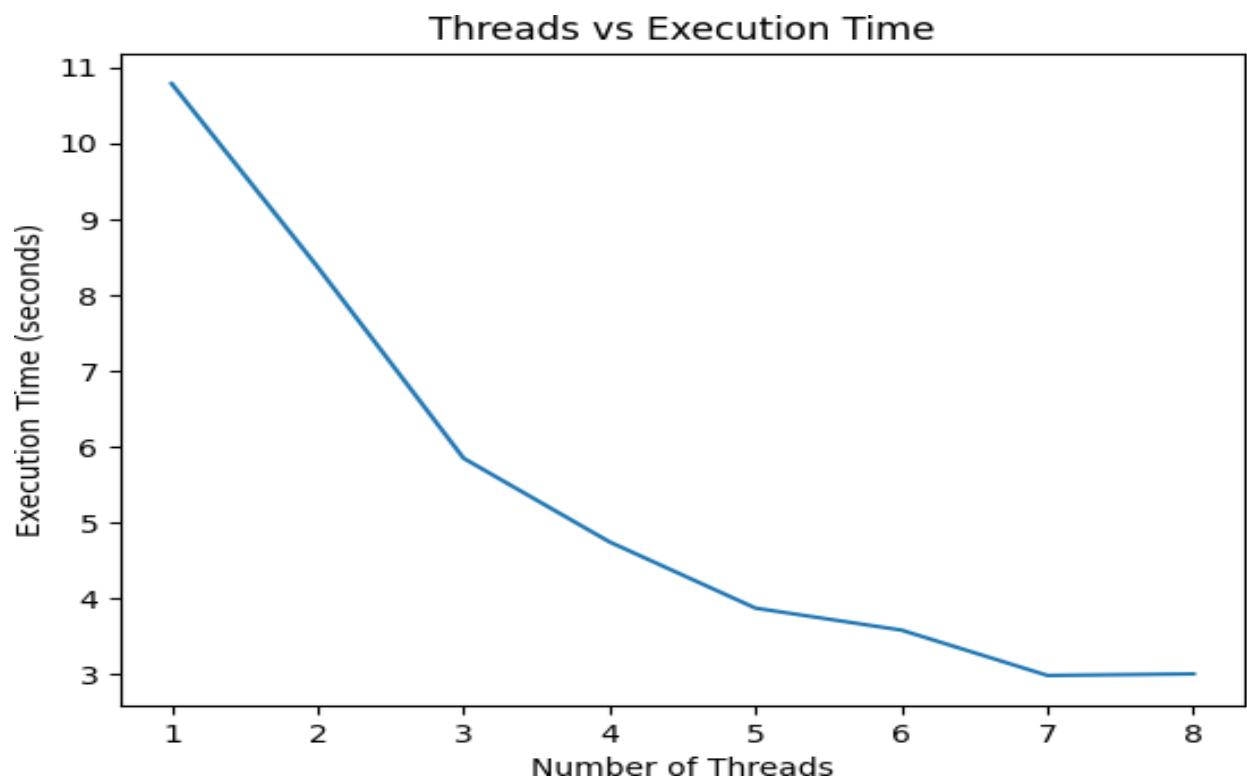
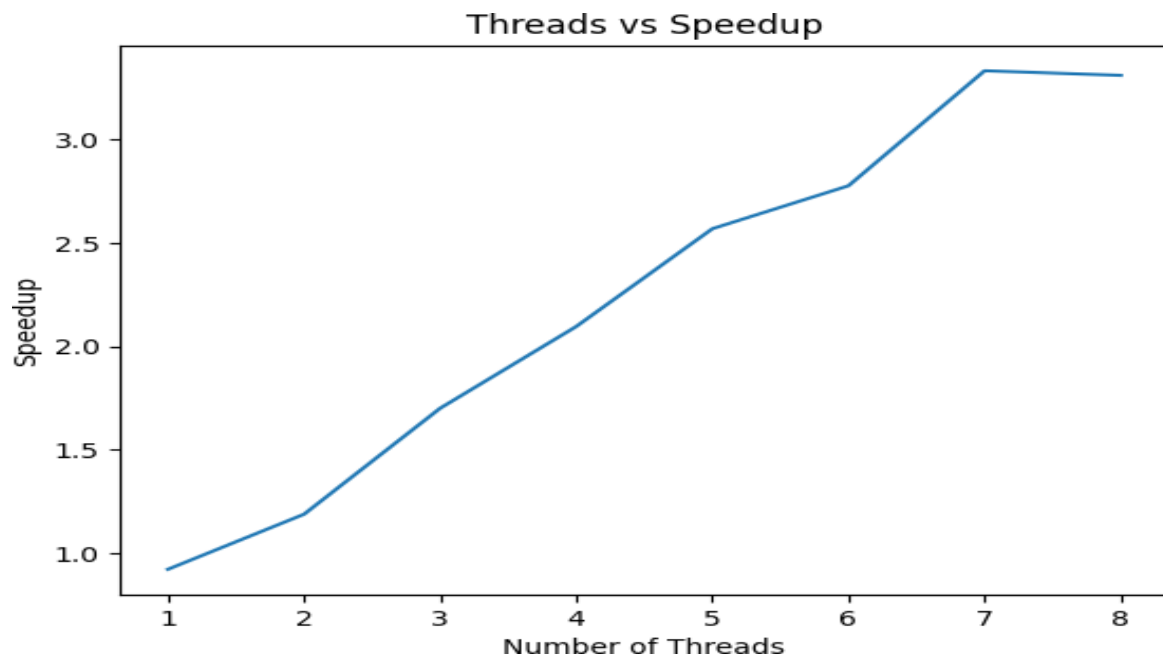
A) FOR MATRIX SIZE 2000x2000

Sequential Time: 9.9948 seconds

TABLE OF RESULTS

Threads	Time (s)	Speedup	Efficiency
1	10.6539	0.938135	0.938135
2	7.97545	1.25320	0.626598
3	5.89134	1.69652	0.565508
4	4.66136	2.14418	0.536046
5	3.87393	2.58002	0.516003
6	3.34312	2.98966	0.498277
7	2.92686	3.41486	0.487837
8	2.59696	3.84866	0.481083

GRAPHS (2000x2000)



CPU Usage

Metric	Value	Description
Task Clock	97,677.07 msec	Total CPU time consumed
CPUs Utilized	1.871	Average cores used
Context Switches	0	No context switching occurred
CPU Migrations	0	No thread migration between CPUs
Page Faults	31,403	Memory page faults
Cycles	155,853,072,309	Total CPU cycles
Instructions	125,151,737,876	Total executed instructions
IPC (Instructions per Cycle)	0.80	CPU efficiency metric
Branches	17,192,255,208	Total branch instructions
Branch Misses	18,292,313	0.11% miss rate
Time Elapsed	52.20 sec	Wall clock time
User Time	97.40 sec	Time spent in user mode
System Time	0.27 sec	Time spent in kernel mode

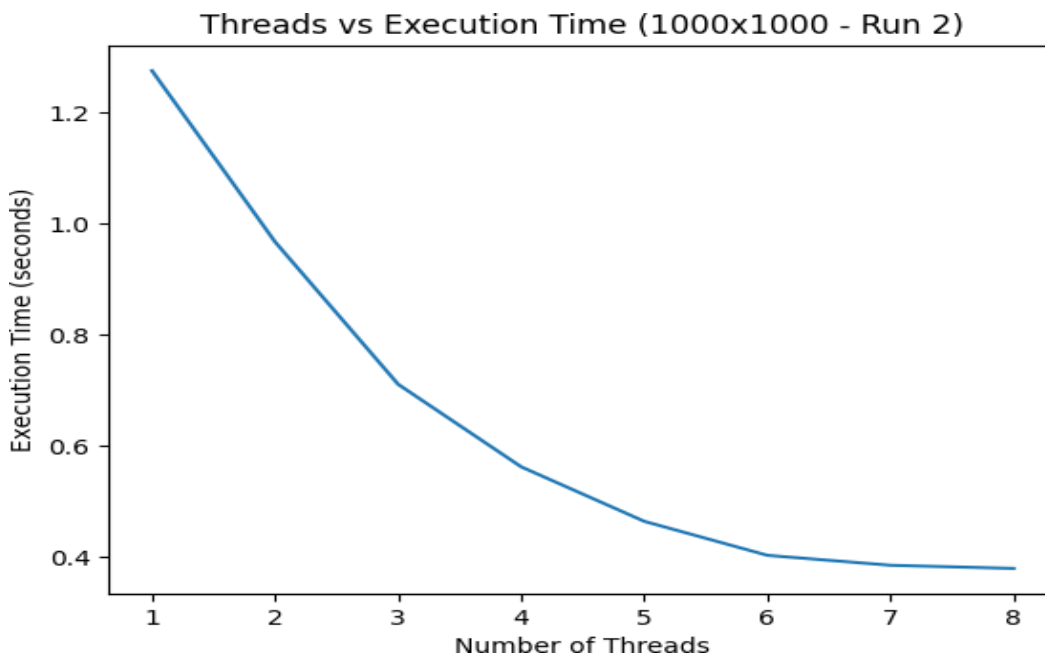
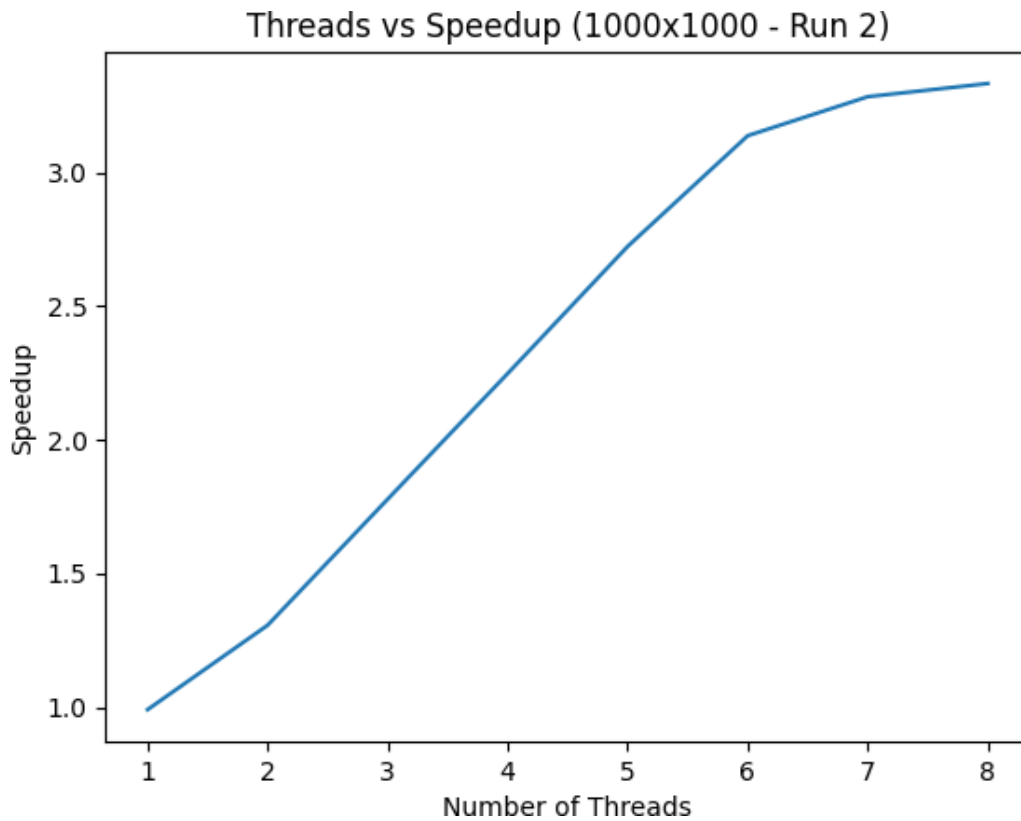
B) FOR MATRIX SIZE 1000x1000

./program 1000 1

Sequential Time: 1.26343 seconds

Threads	Time (s)	Speedup	Efficiency
1	1.27599	0.990153	0.990153
2	0.967545	1.30581	0.652904
3	0.710914	1.77719	0.592396
4	0.562181	2.24737	0.561842
5	0.463998	2.72291	0.544583
6	0.402759	3.13693	0.522822
7	0.384860	3.28282	0.468975
8	0.379164	3.33214	0.416517

GRAPHS (1000x1000)



CPU Usage

Metric	Value	Description
Task Clock	13,199.55 msec	Total CPU time consumed
CPUs Utilized	1.903	Average cores used
Context Switches	0	No context switching
CPU Migrations	0	No thread migration
Page Faults	7,972	Memory page faults
Cycles	19,658,471,015	Total CPU cycles
Instructions	15,866,068,845	Total executed instructions
IPC (Instructions per Cycle)	0.81	CPU execution efficiency
Branches	2,195,182,990	Total branch instructions
Branch Misses	4,613,993	0.21% miss rate
Time Elapsed	6.936 sec	Wall clock time
User Time	12.940 sec	Time in user mode
System Time	0.277 sec	Time in kernel mode

Bottleneck Analysis

The primary performance bottleneck in the correlation computation arises from memory bandwidth limitations rather than arithmetic operations.

Although the algorithm is computationally intensive, it involves repeatedly accessing large rows of the input matrix, making it highly memory-bound.

As the number of threads increases, multiple cores simultaneously access shared memory, leading to cache contention and increased pressure on the memory subsystem.

This reduces scalability at higher thread counts and explains the gradual drop in parallel efficiency.

Additionally, thread management overhead and synchronization costs (even minimal in this case) contribute to diminishing returns beyond a certain number of threads.

Despite effective parallelization and vectorization, the overall performance is ultimately constrained by data movement and cache hierarchy limitations rather than CPU compute capability.

Conclusion

In this assignment, we successfully implemented and optimized a correlation computation program using both sequential and parallel approaches. The objective was to compute the pairwise correlation coefficients between multiple input vectors stored in a matrix format. The solution was first developed as a correct sequential baseline using double-precision arithmetic to ensure numerical accuracy. After verifying correctness, the program was parallelized using OpenMP to exploit multi-core CPU architecture.

The parallelization strategy focused on distributing independent row computations across multiple threads. Since each thread worked on separate portions of the result matrix, no race conditions occurred, ensuring correctness without requiring complex synchronization mechanisms. This demonstrated that the correlation computation problem is well-suited for data parallelism.

Further optimizations were applied to improve performance. These included precomputing means and standard deviations, normalizing rows before computing dot products, improving memory locality through row-wise access, enabling compiler optimizations using `-O3`, and leveraging SIMD vectorization. These improvements significantly reduced redundant computations and improved cache utilization.

Performance evaluation showed that execution time decreased as the number of threads increased. Near-linear speedup was observed at lower thread counts, indicating effective parallelization. However, as the number of threads increased further, efficiency gradually decreased. This behavior is expected due to practical hardware limitations such as memory bandwidth constraints, cache contention, and thread management overhead. The analysis highlights that while computational workload can scale well, memory-intensive applications

eventually become bound by memory subsystem performance rather than CPU arithmetic capability.

Overall, this assignment demonstrated the practical application of parallel programming concepts, performance measurement, and optimization techniques in high-performance computing. It reinforced the importance of starting with a correct baseline implementation, carefully introducing parallelism, and analyzing scalability using quantitative metrics such as speedup and efficiency. The results confirm that parallel computing, when combined with thoughtful optimization, can achieve substantial performance gains while revealing real-world architectural limitations